How IoC Container Works Internally

The lifecycle of a Spring IoC container can be broken into distinct phases:

1. **Resource Loading and BeanDefinition Creation**
    - The container loads configuration sources (XML files, @Configuration classes, annotated components).
    - Each bean declaration is parsed into a BeanDefinition object – metadata containing: class name, scope, constructor arguments, property values, init/destroy method names, etc.
    - For annotation-based configuration, component scanning converts @Componentstereotyped classes into BeanDefinition objects.
2. **BeanFactoryPostProcessor Execution**
    - Before beans are instantiated, the container invokes all registered BeanFactoryPostProcessor beans.

- These can modify the BeanDefinition objects. For
  example, PropertySourcesPlaceholderConfigurer replaces ${...} placeholders,
  and ConfigurationClassPostProcessor processes @Configuration classes and @Bean methods, generating additional BeanDefinition entries.

- This phase allows the container to “prepare” its configuration before actual object creation.

3. **Bean Instantiation and Dependency Resolution**

- The container identifies dependencies from the BeanDefinition (constructor args, property refs).

- When a bean is first requested (or eagerly at startup for ApplicationContext), the container constructs a dependency graph, resolves all required references, and recursively creates beans if they don’t already exist.

- Instantiation can happen via reflection (Class.newInstance()), factory method, or an InstanceSupplier (Spring 5+).

- Circular dependencies are resolved using pre-allocated object references (singletons only); constructor-based circular dependencies will fail.

4. **BeanPostProcessor Processing**

- After a bean instance is created and dependencies are injected, the container runs it through all registered BeanPostProcessor beans.
- BeanPostProcessor can wrap the bean with a proxy or perform additional configuration. For example:
  - AutowiredAnnotationBeanPostProcessor handles @Autowired and @Value.
  - CommonAnnotationBeanPostProcessor supports @PostConstruct and @PreDestroy.
  - AOP proxies are applied here if the bean matches an advisor.
- This two-stage processing (postProcessBeforeInitialization and postProcessAfterInitialization) allows full customization.

5. **Initialization Callbacks**
    - After postProcessBeforeInitialization, the container
      invokes afterPropertiesSet() (from InitializingBean) and custom init methods
      (e.g., @PostConstruct).
    - Beans perform resource setup, cache pre-warming, etc., at this stage.
6. **Ready for Use**
    - The fully initialized bean is placed in the singleton cache (if singleton scope) and returned to the caller.
7. **Container Destruction**
    - On close() (for ApplicationContext), the container destroys singletons.
    - It calls destroy() (from DisposableBean) or @PreDestroy methods, releasing resources.

Internally, the container uses several key components:

- DefaultListableBeanFactory: the core registry that holds bean definitions and resolves dependencies.

- AbstractAutowireCapableBeanFactory: handles bean creation, property population, and BeanPostProcessor hooks.
- AbstractApplicationContext: orchestrates the refresh lifecycle (refresh(), etc.).